

Dithering su GPU tramite OpenCL

Giovanni Maria Manduca

Dipartimento di Matematica e Informatica / Università degli Studi di Catania

30 giugno 2026

Sommario

Questo report presenta i risultati dell'implementazione di algoritmi di dithering accelerati su GPU tramite OpenCL. Vengono confrontate diverse tecniche, tra cui thresholding, random dithering, Ordered Dithering (con matrici di Bayer di dimensione crescente) ed error diffusion (Floyd-Steinberg). Oltre ai risultati, il report contiene anche le limitazioni e le sfide implementative specifiche della GPGPU.

1 Preliminari all'implementazione

1.1 Cos'è il dithering

Il dithering può essere interpretato come un insieme di tecniche per ridurre gli artefatti di quantizzazione in segnali discreti, introducendo variazioni controllate nel dominio spaziale.

Il thresholding non produce risultati visivamente gradevoli, specialmente per gradienti e cambi di luminanza non improvvisi, producendo zone a tinta unita (chiamate anche artefatti di banding).

1.2.2 Dithering randomizzato

Nel dithering randomizzato, il segnale viene perturbato prima della quantizzazione:

$$I'(x, y) = Q(I(x, y) + n(x, y)) \quad (3)$$

dove il rumore è tipicamente uniforme:

$$n(x, y) \sim \mathcal{U}(-\Delta, \Delta) \quad (4)$$

con:

$$\Delta = \frac{s}{2} = \frac{255}{2(L-1)} \quad (5)$$

Questo essenzialmente fa sì che un qualsiasi valore possa essere quantizzato al valore di quantizzazione immediatamente superiore o inferiore, con una probabilità che dipende dalla vicinanza (o distanza) dal valore di quantizzazione stesso. Il dithering randomizzato, per quanto semplice, funziona sorprendentemente bene rispetto al thresholding.

1.2.3 Dithering ordinato (matrici di Bayer)

Nel dithering ordinato si introduce un offset deterministico periodico:

$$I'(x, y) = Q(I(x, y) + B'(x, y)) \quad (6)$$

dove $B'(x, y)$ è una matrice periodica con valori nell'intervallo:

$$B'(x, y) \in \left[-\frac{s}{2}, \frac{s}{2}\right] \quad (7)$$

ottenuta a partire dalla matrice di Bayer "grezza" $M(x, y)$ (a valori interi $0, 1, \dots, n^2 - 1$, dove $n \times n$ è la dimensione della matrice), normalizzata su $[0, 1]$ e poi centrata e scalata:

$$B'(x, y) = s \cdot \left(\frac{M(x, y)}{n^2} - \frac{1}{2} \right) \quad (8)$$

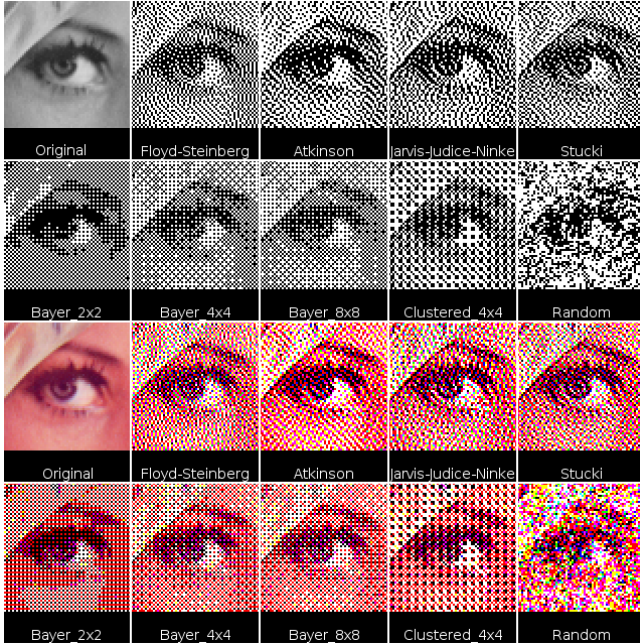


Figura 1: Esempio di output di varie tecniche di dithering su una sezione di Lena

1.2 Tecniche di dithering

1.2.1 Thresholding (quantizzazione lineare)

Sia L il numero di livelli di quantizzazione desiderato. Il passo è definito come:

$$s = \frac{255}{L-1}, \quad \alpha = \frac{1}{s} \quad (1)$$

La quantizzazione uniforme è:

$$Q(x) = \text{clamp}(\text{round}(x \cdot \alpha) \cdot s, 0, 255) \quad (2)$$

Definizione ricorsiva. La matrice di Bayer grezza M_n di dimensione $n \times n$ (con $n = 2^k$) si costruisce ricorsivamente a partire dal caso base 2×2 :

$$M_2 = \begin{pmatrix} 0 & 2 \\ 3 & 1 \end{pmatrix} \quad (9)$$

Il passo ricorsivo genera M_{2n} a partire da M_n tramite la formula:

$$M_{2n} = \begin{pmatrix} 4M_n & 4M_n + 2 \\ 4M_n + 3 & 4M_n + 1 \end{pmatrix} \quad (10)$$

dove ciascun blocco ha dimensione $n \times n$ e $4M_n$ indica la matrice M_n moltiplicata elemento per elemento per 4. Ad esempio, da M_2 si ottiene M_4 :

$$M_4 = \begin{pmatrix} 0 & 8 & 2 & 10 \\ 12 & 4 & 14 & 6 \\ 3 & 11 & 1 & 9 \\ 15 & 7 & 13 & 5 \end{pmatrix} \quad (11)$$

Nell'implementazione dei kernel, partiamo dai valori della matrice grezza per tornare a quelli regolarizzati tramite un'opportuna trasformazione lineare.

La struttura periodica consente una distribuzione regolare dell'errore senza dipendenze tra pixel. Vedremo che questo lo rende imbarazzantemente parallelizzabile su GPU.

1.2.4 Dithering tramite diffusione dell'errore (Floyd-Steinberg)

I metodi di dithering tramite diffusione dell'errore cercano di usare la quantizzazione dei pixel circostanti per correggere ulteriormente il processo di quantizzazione del singolo pixel. Questo migliora significativamente la qualità percepita rispetto ad altre tecniche, ma introduce una dipendenza spaziale tra pixel adiacenti, rendendo la parallelizzazione non banale.

Sia $I(x, y)$ l'intensità del pixel in ingresso e $I'(x, y)$ il valore quantizzato. La quantizzazione è definita come:

$$I'(x, y) = Q(I(x, y) + E(x, y)) \quad (12)$$

dove $Q(\cdot)$ è la funzione di quantizzazione uniforme a L livelli e $E(x, y)$ rappresenta l'errore accumulato dai pixel precedentemente processati.

L'errore locale di quantizzazione è definito come:

$$e(x, y) = I(x, y) - I'(x, y) \quad (13)$$

Tale errore viene ridistribuito ai pixel vicini non ancora processati secondo uno schema pesato:

$$\begin{aligned} I(x+1, y) &+= e(x, y) \cdot \frac{7}{16} \\ I(x-1, y+1) &+= e(x, y) \cdot \frac{3}{16} \\ I(x, y+1) &+= e(x, y) \cdot \frac{5}{16} \\ I(x+1, y+1) &+= e(x, y) \cdot \frac{1}{16} \end{aligned} \quad (14)$$

1.3 Panoramica su OpenCL

OpenCL è un framework che permette lo sviluppo e l'esecuzione di programmi capaci di essere eseguiti su dispositivi eterogenei come CPU e GPU. Il modello di esecuzione è basato su kernel che operano su uno spazio di lavoro (potenzialmente multidimensionale), dove ogni work-item esegue lo stesso codice su dati differenti (seguendo un approccio SIMD/SIMT).

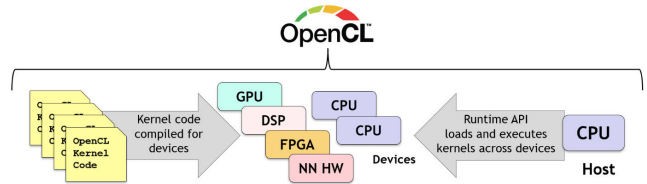


Figura 2: Modello di esecuzione di OpenCL: il codice dei kernel è separato dall'API di runtime, che compila e lancia dinamicamente i kernel.

2 Implementazione

Seguono i dettagli implementativi rilevanti del codice di progetto.

2.1 Baseline CPU

Per poter confrontare l'efficienza di esecuzione degli algoritmi, sono state implementate delle baseline di dithering single-threaded per CPU. È bene notare che, mentre i kernel paralleli OpenCL accettano in input immagini RGBA, la baseline CPU funziona su immagini RGB (quindi processa un canale in meno).

L'eseguibile compilato permette il lancio dell'esecuzione di uno qualsiasi degli algoritmi in esame su un'immagine e il salvataggio dei risultati. Inoltre, viene fornita la misura del tempo richiesto dall'esecuzione dell'algoritmo (escludendo quindi caricamento e salvataggio dell'immagine).

Per facilitare il caricamento e salvataggio delle immagini da disco, si è fatto uso della libreria png.h. È stata inoltre implementata una modalità di benchmark che genera un'immagine randomica di lato definito tramite argomento di lancio ed esegue il dithering sulla stessa.

2.2 Codice per il lancio dei kernel su GPU

Il programma gestisce il lancio di due gruppi di algoritmi di dithering con strategie di esecuzione OpenCL nettamente diverse.

Dithering ordinato. Per gli algoritmi `threshold`, `random` e `bayer2/4/8`, la funzione `run_parallel_dither` lancia un kernel su un `NDRange` bidimensionale pari alle dimensioni dell'immagine:

$$\text{gws} = (\text{width}, \text{height}) \quad (15)$$

senza esplicitare un local work size. Ogni pixel è elaborato in modo indipendente dagli altri, essendo di fatto l'algoritmo imbarazzantemente parallelo.

Floyd-Steinberg. La funzione `run_floyd_steinberg` utilizza invece un `NDRange` monodimensionale composto da un *singolo work-group*:

$$\text{lws} = \text{gws} = \text{block_size} \quad (16)$$

dove `block_size` è il valore di altezza dell'immagine, eventualmente ridotto al limite `CL_KERNEL_WORK_GROUP_SIZE` del kernel compilato.

2.3 Derivazione di un algoritmo parallelo per Floyd-Steinberg

A prima vista, l'algoritmo di Floyd-Steinberg sembra intrinsecamente sequenziale: ogni pixel utilizza nel proprio calcolo l'errore di quantizzazione propagato dai pixel precedenti, suggerendo un'unica catena di dipendenze che attraversa l'intera immagine pixel per pixel.

2.3.1 Analisi delle dipendenze.

Un'analisi più attenta rivela che questa intuizione è eccessivamente pessimistica. La diffusione dell'errore di Floyd-Steinberg propaga il valore residuo del pixel (x, y) a un intorno immediato, ma espandendo ricorsivamente queste dipendenze, si osserva che il pixel (x, y) dipende, in ultima analisi, esclusivamente dai pixel contenuti nel *triangolo rettangolo isoscele* con angolo retto nel pixel in alto a sinistra dell'immagine (coordinate $(0, 0)$) e avente (x, y) sull'ipotenusa, troncato verticalmente all'altezza del pixel stesso. In altre parole, un pixel non dipende da *tutta* l'immagine già processata, ma soltanto dalla regione raggiungibile propagando l'errore in diagonale verso il basso a destra fino a raggiungere la sua stessa riga e colonna.

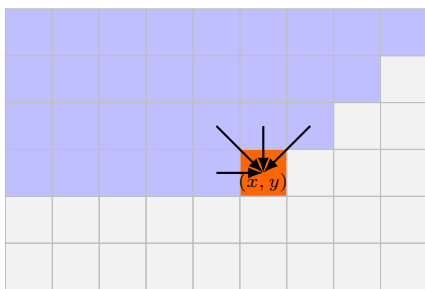


Figura 3: Dipendenze del pixel evidenziato (x, y) (in arancione). In blu, l'insieme delle dipendenze del pixel. Le frecce indicano le quattro dipendenze immediate.

2.3.2 Lancio con fronte diagonale.

Sfruttando questa proprietà, possiamo organizzare il calcolo come un fronte d'onda diagonale che avanza dall'angolo in alto a sinistra verso il basso a destra: a ogni passo k fanno parte della diagonale tutti i pixel che rispettano $x + y = k$, per $k = 0, 1, \dots, (W - 1) + (H - 1)$. In

particolare, le dipendenze di ogni diagonale sono esattamente i pixel contenuti nel triangolo superiore limitato dalla diagonale stessa.

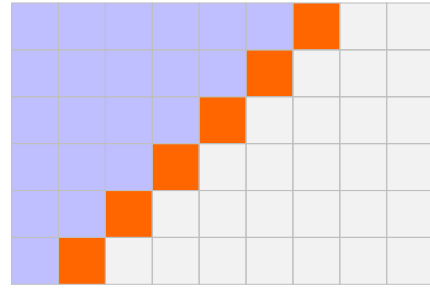


Figura 4: Fronte diagonale teorico: in arancione, l'unica anti-diagonale attiva al passo $k = 6$, i cui elementi dipendono solo dagli elementi appartenenti al triangolo. In blu, il triangolo superiore contenente tutte le dipendenze accumulate dai passi precedenti.

2.3.3 Dipendenze sulla diagonale, scelta del work-group e del delay.

Affinché un work-item alla diagonale k possa eseguire correttamente il proprio calcolo, deve attendere che i work-item delle sue dipendenze (Figura 3) abbiano completato e reso visibile la scrittura del proprio errore residuo. Purtroppo, questo include anche altri pixel sulla diagonale, e in particolare, lungo la diagonale stessa, le dipendenze sono concatenate piuttosto che immediate (l'ultimo pixel della diagonale ha bisogno che tutti i pixel precedenti abbiano già completato la loro propagazione). Per evitare race condition, possiamo utilizzare un delay, effettivamente usando una diagonale con coefficiente angolare diverso da 1. Risolto questo primo problema, bobbiamo comunque assicurarci di aspettare il completamento della diagonale precedente (che contiene dipendenze per ogni singolo pixel della diagonale). Poiché OpenCL non offre un meccanismo di sincronizzazione globale tra work-group distinti all'interno di un singolo kernel, l'unico modo per garantire questa coordinazione *a ogni diagonale* è utilizzare una `barrier()` su tutti gli elementi contemporaneamente attivi, il che richiede l'uso di un *singolo work-group*.

Determiniamo ora il *delay* minimo, ossia di quante diagonali un work-item deve restare "indietro" rispetto al work-item della riga sopra per garantire la correttezza:

- **Delay = 3.** Richiede una sola `barrier()` per diagonale: lo skew del fronte d'onda assicura l'impossibilità di race condition tra i work-item del kernel (figura 5).
- **Delay = 2.** È raggiungibile introducendo una *barriera extra* dedicata specificamente alla dipendenza "bottom-left" (figura 6).

In entrambi i casi, la necessità di almeno una barriera condivisa da *tutti* gli elementi attivi sulla diagonale conferma che l'intera computazione deve risiedere in un singolo work-group: è l'unica unità di esecuzione OpenCL per cui `barrier()` garantisce una sincronizzazione

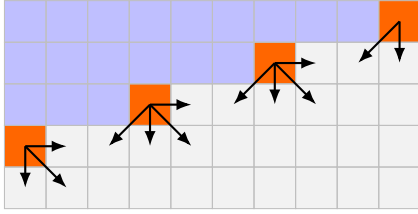


Figura 5: Fronte di propagazione per delay = 3: le frecce rappresentano la propagazione dell'errore delle celle attive correnti (in arancione). Notare come nessuna operazione di scrittura si incrocia (no race condition).

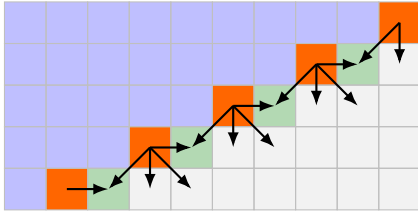


Figura 6: Fronte di propagazione per delay = 2: le frecce escono da ciascun pixel attivo (arancione) nelle quattro direzioni di Floyd-Steinberg. Due di esse si incontrano sulle celle di race condition (verde), gestite tramite una barriera extra dedicata; le altre due raggiungono celle senza conflitti.

globale e coerente della memoria locale/globale tra tutti i work-item coinvolti.

2.3.4 Artefatti derivanti da gestione incorretta del fronte di propagazione

Durante lo sviluppo, è stato interessante analizzare artefatti caratteristici derivanti dall'uso incorretto del delay e/o delle barriere.

Rimozione di una barriera. Prendendo in considerazione il kernel con delay pari a 2, rimuovere una qualsiasi barriera produce risultati non deterministici, ma non visivamente impattati (all'occhio umano).

Rimozione di tutte le barriere. Combinando invece la rimozione di entrambe le barriere, si ottengono dei caratteristici artefatti orizzontali (figura 7) ogni 32 pixel. Questo numero non è casuale: l'hardware su cui sono eseguiti questi test è una GPU NVIDIA RTX 3080. In particolare, il warp size di questa GPU è esattamente pari a 32, il che vuol dire che il motivo degli artefatti a riga è proprio la desincronizzazione tra i vari warp. Rimuovendo infatti entrambe le barriere, non c'è alcuna garanzia su un upper bound dell'errore di ordine di esecuzione (l'ultimo warp in verticale potrebbe finire prima di qualsiasi altro).

2.4 Vettorizzazione del kernel di quantizzazione

OpenCL fornisce nativamente dei tipi vettorizzati come `float4`. Possiamo usarli per processare l'immagine come una matrice $m \times n$ piuttosto che $c \times m \times n$. La



Figura 7: Artefatti di sincronizzazione dei warp osservati in assenza di alcuna barriera (linee orizzontali particolarmente visibili nelle regioni scure).

vettorizzazione, applicata alla funzione di quantizzazione del kernel di Floyd-Steinberg, produce un eseguibile quasi 4 volte più veloce rispetto all'iterazione sui canali.

2.4.1 Altre micro ottimizzazioni

Il progetto contiene versioni progressivamente ottimizzate del kernel di Floyd-Steinberg. In particolare, oltre alla vettorizzazione, sono state implementate:

- **Hardcoding dei pesi di Floyd-Steinberg**, di modo da evitare iterazioni e branching sul valore della matrice di propagazione.
- **Caching dei valori di step e step inverso**, di modo da saltare alcune operazioni aritmetiche.
- **Rimozione della divisione float dalla funzione di quantizzazione**, a favore del prodotto per l'inverso dello step, operazione di floating-point più rapida.

3 Risultati

3.1 Valutazione Qualitativa

Visivamente, tutti i kernel performano come previsto, producendo immagini correttamente ditherizzate. Il codice fornito può essere facilmente usato per valutare i risultati. Tutte le implementazioni fornite producono pattern di dithering strettamente deterministici (a meno del seeding per il random dithering).

3.2 Prestazioni

In questa sezione sono dettagliati i guadagni di performance rispetto all'implementazione strettamente seriale. Gli script per riprodurre le simulazioni sono forniti nella cartella di script. Tutti i risultati sono computati con una run di warmup e cinque di raccolta dati.

3.2.1 Algoritmi imbarazzantemente paralleli

I guadagni sugli algoritmi imbarazzantemente paralleli sono enormi (nonostante non siano state usate tecniche come la vettorizzazione), in quanto i kernel trasformano un problema di complessità $\text{width} \times \text{height}$ in uno di complessità $\text{width} \times \text{height} / \text{compute_units}$, potenzialmente $O(1)$ (figure 9, 10, 11, 12, 13).

Possiamo inoltre notare come, fino a quando il numero totale dei pixel rimane minore o simile a quello delle unità di compute, i timing si tengono essenzialmente uguali, indipendentemente dalla dimensione dell'immagine.

3.2.2 Floyd-Steinberg

Sebbene sia possibile velocizzare anche questo metodo, i guadagni sono notevolmente inferiori rispetto ai precedenti, date le limitazioni di parallelizzazione derivate dalla necessità dell'uso di un singolo work-group. Il miglior risultato ottenuto si avvicina a uno speedup $4 \times$ (figura 14). La GPU usata ha infatti 8704 CUDA core, mentre noi ne stiamo usando soltanto 256, in quanto è l'unità massima sulla quale possiamo usare barriere.

4 Discussione

È evidente che le tecniche con molte dipendenze sequenziali non siano adatte all'esecuzione parallela, ma possono comunque essere velocizzate prendendo appropriati accorgimenti.

4.1 Applicazioni moderne dell'ordered dithering

Il vero vincitore resta comunque l'ordered dithering, che offre comunque una qualità d'immagine nettamente superiore al thresholding, con tempi di esecuzione dell'ordine di pochi microsecondi su immagini sufficientemente grandi, nonostante non abbiamo implementato molte delle ulteriori ottimizzazioni possibili (essendomi concentrato sulla sfida maggiore, ovvero ottimizzare il Floyd-Steinberg).

L'ordered dithering è infatti usato tutt'oggi in tecniche come il transparency dithering (Figura 8) in diversi motori di rendering e videogiochi, in quanto offre alcuni vantaggi di performance (non bisogna leggere il valore corrente del pixel dal frame buffer) e comodità di implementazione (invarianza rispetto all'ordine dei poligoni) se comparato all'alpha blending, a costo dell'introduzione dei pattern tipici delle matrici di Bayer (che comunque spesso non si notano su immagini grandi e/o soggetti in movimento).

4.2 Sviluppi futuri per Floyd-Steinberg

La natura inerentemente sequenziale di Floyd-Steinberg rende difficile l'uso di molte tecniche di ottimizzazione tipiche della GPGPU (come l'uso di *texture*, *local memory*, etc...).

Ciononostante, non sono da escludere possibilità di miglioramento: sicuramente interessante indagare quanto questi constraint di dipendenza sequenziale possano

essere rilassati senza inficiare la qualità caratteristica dell'algoritmo, limitando ad esempio la distanza dalla quale un pixel può partecipare alla propagazione dell'errore o eseguendo l'algoritmo su piccoli chunk (ad esempio di dimensione pari a quella del *warp*) e "fondendo" i risultati tramite appositi algoritmi di "merging" del dithering.

Un'altra area di possibile esplorazione è quella di spingere ulteriormente la vettorizzazione (che nella mia implementazione viene usata solo per i canali colore) al processing di pixel adiacenti. Le relazioni di dipendenza si complicano molto velocemente, ma potrebbe comunque valere la pena modellarle (pagando il costo di branch aggiuntivi e operazioni che gestiscono le dipendenze all'interno del gruppo di pixel adiacenti) per dimezzare (*float8*) o dividere per quattro (*float16*) il numero di iterazioni necessarie.

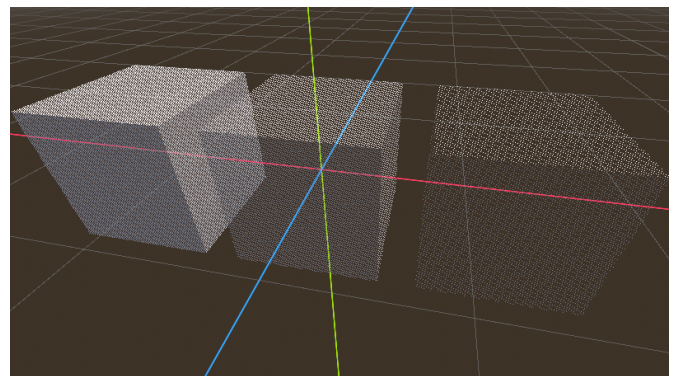
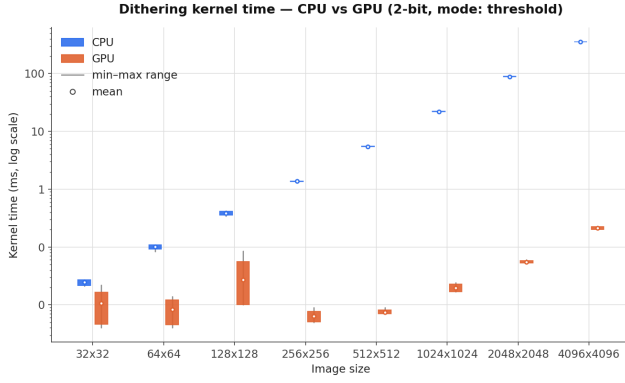


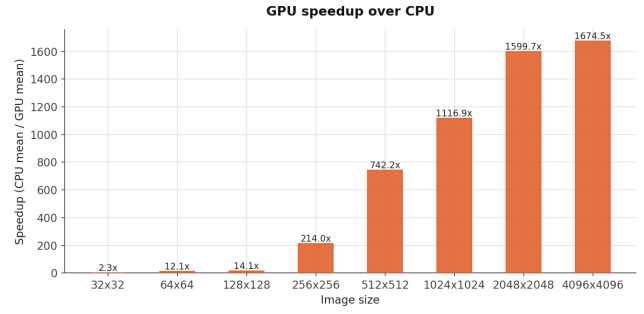
Figura 8: Esempio di uno shader che usa l'ordered dithering per simulare la trasparenza su Unity.

5 Conclusioni

Questa relazione presenta il problema del dithering nell'ambito della programmazione parallela. I programmi sviluppati tramite OpenCL implementano efficientemente gli algoritmi di dithering tramite kernel, permettendone l'esecuzione parallela, ottenendo prestazioni adatte all'applicazione in tempo reale per alcuni algoritmi, e mostrando comunque un miglioramento su tutti gli algoritmi testati rispetto all'esecuzione sequenziale su CPU.

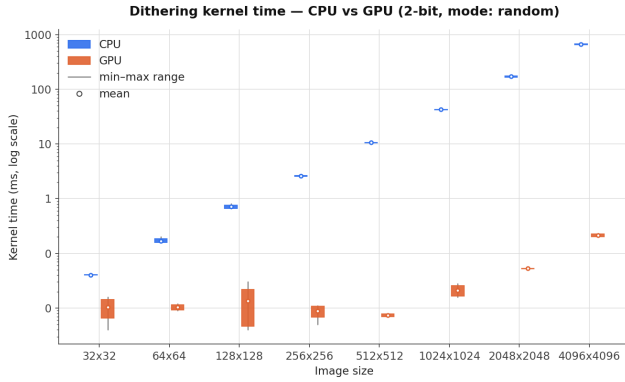


(a) Timing

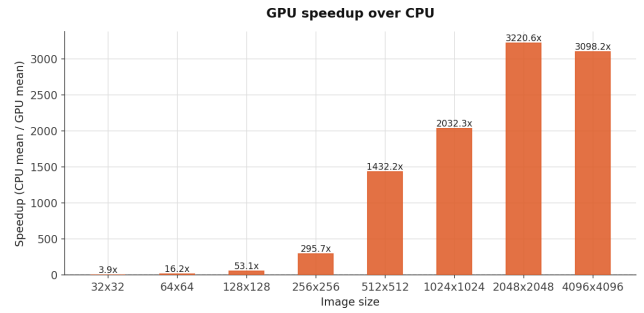


(b) Speedup

Figure 9: Thresholding

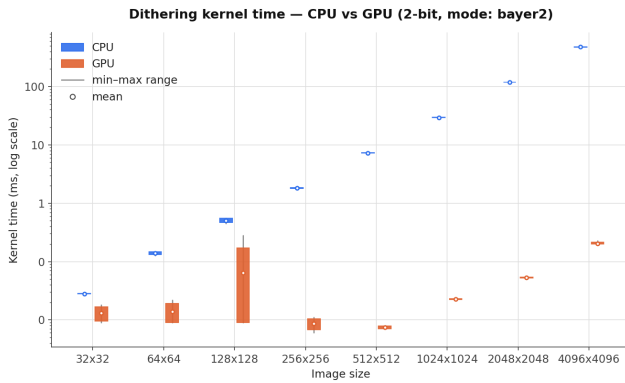


(a) Timing

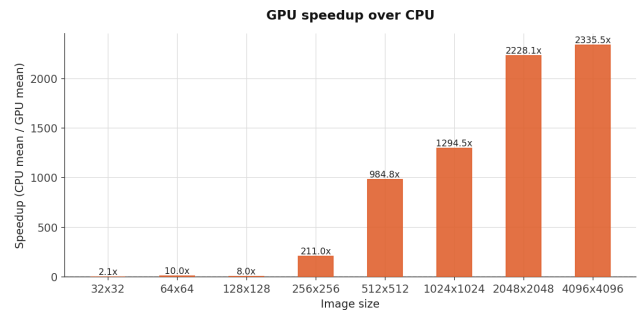


(b) Speedup

Figure 10: Random dithering

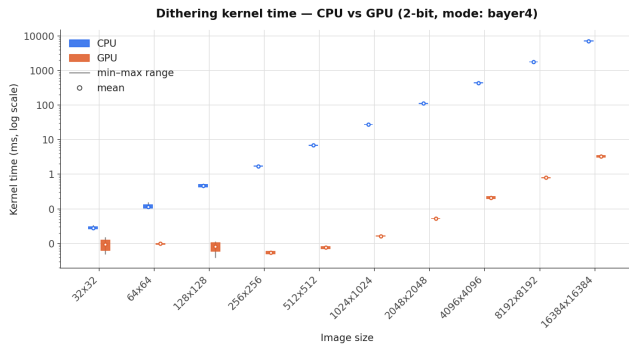


(a) Timing

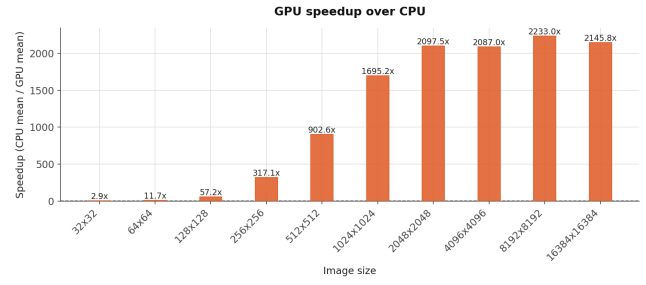


(b) Speedup

Figure 11: Ordered dithering (Bayer 2×2)

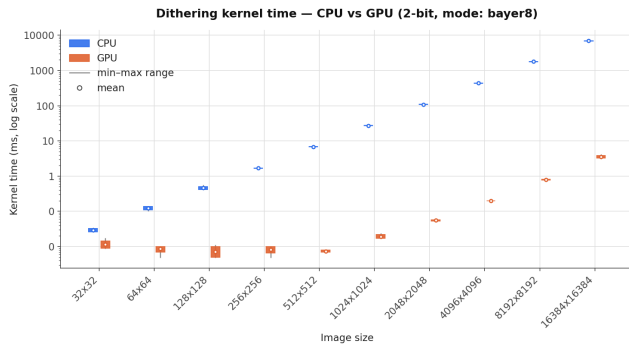


(a) Timing

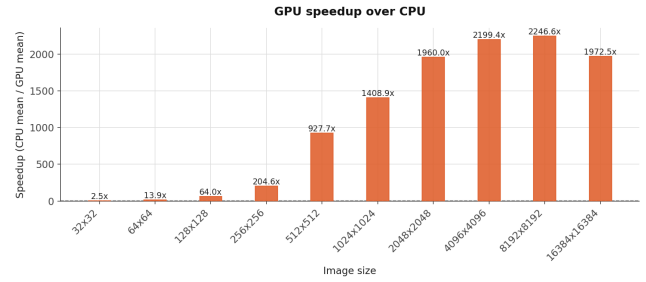


(b) Speedup

Figure 12: Ordered dithering (Bayer 4×4)

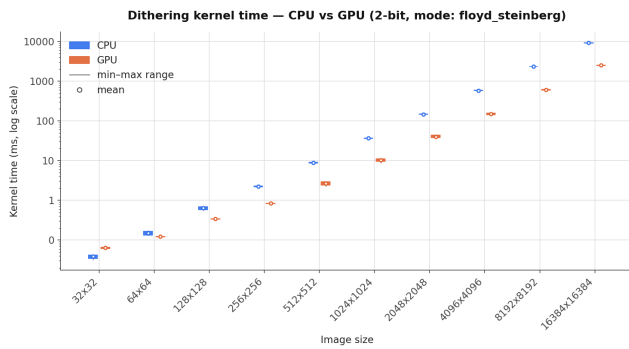


(a) Timing

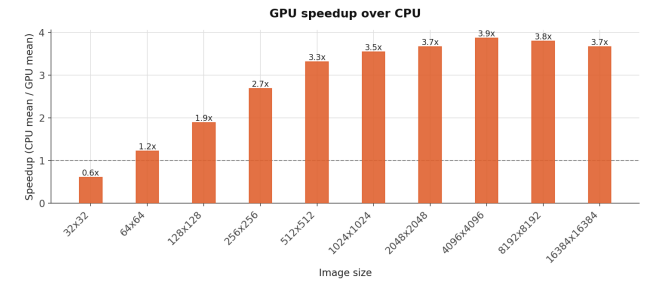


(b) Speedup

Figure 13: Ordered dithering (Bayer 8×8)



(a) Timing



(b) Speedup

Figure 14: Floyd-Steinberg dithering